

Лекция 6. Алгоритмы поиска MST.
Центроидная декомпозиция. Heavy-light
декомпозиция

1 Минимальные остовные деревья. Терминология и лемма о безопасном ребре

Определение 1.1. Остовом графа $G = (V, E)$ будем называть граф $H = (V, E')$, где $E' \subseteq E$.

Определение 1.2. Остовным деревом графа $G = (V, E)$ будем называть остов, образующий дерево.

Определение 1.3. Минимальным остовным деревом (*minimum spanning tree, MST*) графа $G = (V, E, w)$ будем называть такое остовное дерево $H = (V, E', w)$, что

$$\sum_{e \in E'} w(e) \rightarrow \min$$

Определение 1.4. Пару множеств вершин $\langle S, T \rangle$ будем называть *разрезом*, если $S \cup T = V, S \cap T = \emptyset$.

Определение 1.5. Будем говорить, что ребро $(u, v) \in E$ *пересекает разрез* $\langle S, T \rangle$, если u и v лежат в разных частях разреза (в разных множествах).

Определение 1.6. Пусть $G' = (V, E')$ — подграф некоторого MST G . Будем говорить, что ребро $(u, v) \notin E'$ *безопасное*, если при добавлении его в G' , $G' \cup \{(u, v)\}$ также является подграфом некоторого MST графа G .

Лемма 1.7 (о безопасном ребре). Рассмотрим связный взвешенный неориентированный граф $G = (V, E, w)$, $w : E \rightarrow \mathbb{R}$. Пусть $G' = (V, E')$ — подграф некоторого MST графа G , $\langle S, T \rangle$ — разрез G , такой, что ни одно ребро из E' не пересекает разрез, а (u, v) — ребро минимального веса, пересекающее разрез $\langle S, T \rangle$. Тогда ребро $e = (u, v)$ является безопасным для G' .

Замечание 1.8 (неформальное пояснение формулировки леммы). Представьте, что для некоторого MST графа G мы выбрали произвольный разрез $\langle S, T \rangle$ и удалили ребро, пересекающее этот разрез, из MST. Тогда это ребро можно восстановить, выбрав из всех рёбер, пересекающих этот разрез, ребро минимального веса.

Доказательство. Построим E' до некоторого MST, обозначим его $T_{\min}$. Если ребро $e \in T_{\min}$, то лемма доказана, поэтому пусть $e \notin T_{\min}$. Рассмотрим путь в $T_{\min}$ от вершины $u \in S$ до вершины $v \in T$. Хотя бы одно ребро на этом пути пересекает разрез, назовём его e' . По условию леммы $w(e) \leq w(e')$. Заменим ребро e' в $T_{\min}$ на ребро e . Полученное дерево также является MST графа G , поскольку полученный граф остаётся связным и ациклическим, то есть деревом, и вес дерева не увеличился. Следовательно, $E' \cup \{e\}$ можно дополнить до минимального остовного дерева в графе G , то есть ребро e — безопасное.

Пояснение: логично, рассмотрим MST, содержащее E' . Тогда если заменить ребро, пересекающее разрез $\langle S, T \rangle$ на e , то граф всё ещё останется связным и вес дерева не увеличится, а значит, что e принадлежит некоторому MST, в которое включено E' . $\square$

2 Алгоритм Прима

Введение. Теория — это, конечно, хорошо, но как искать MST для произвольного взвешенного графа? В этом параграфе будет рассмотрен первый алгоритм поиска MST, который по структуре очень напоминает алгоритм Дейкстры.

Алгоритм 2.1 (Прима). Пусть $G = (V, E, w)$, $w : E \rightarrow \mathbb{R}$ — связный взвешенный неориентированный граф. Алгоритм поддерживает множество S — множество вершин, для которого MST уже найдено. Алгоритм расширяет S , добавляя на очередной итерации безопасное ребро в разрезе $\langle S, (V \setminus S) \rangle$. Изначально $S = \{s\}$, где s — произвольная вершина.

1. Рассмотрим разрез $\langle S, (V \setminus S) \rangle$. Найдём безопасное для него ребро $e = (u, v)$, где $u \in S$.
2. Добавим e в миностов и v в S .
3. Добавим в множество рёбер, пересекающих разрез, рёбра, выходящие из v и идущие не в S .
4. Пока $S \neq V$, повторять шаги 1-3.

Чтобы эффективно находить безопасное ребро для разреза, воспользуемся леммой о безопасном ребре. Это должно быть ребро наименьшего веса, пересекающее разрез. Поэтому удобно поддерживать множество рёбер, пересекающих текущий разрез.

Замечание 2.2. Корректность алгоритма Прима напрямую следует из леммы о безопасном ребре. На каждом шаге алгоритма добавляется очередное безопасное ребро.

Замечание 2.3 (время работы). Алгоритм Прима делает:

- Добавление рёбер в множество пересекающих разрез $|E|$ раз;
- Удаление ребра наименьшего веса, пересекающего разрез $(|V| - 1)$ раз.

Из этого следует уже известная таблица асимптотик поддержания этих двух операций: Добавление в

Контейнер рёбер	Добавление	Извлечение	Итого
Массив	$O(1)$	$O(V)$	$O(V ^2 + E)$
Двоичная куча	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
Фибоначчиева куча	$O^*(1)$	$O(\log V)$	$O(E + V \log V)$

массиве можно реализовать так: изначально вес равен ∞ , а далее, при пересечении разреза, ставить им реальный вес.

Замечание 2.4. Начиная с этого алгоритма и далее, по мере усложнения теории, написанию кода уделяется всё меньше внимания. Наши конспекты же являются исключением, автор постарается написать рабочий код везде, где это только возможно.

Псевдокод (массив). Приведём реализацию алгоритма Прима за $O(|V|^2 + |E|)$ на массиве. Эта реализация полезна, например, когда дан большой плотный граф (особенно где рёбра заданы неявно). Например, если даны точки на плоскости (или в пространстве) с евклидовой метрикой. Следующая реализация упрощённая, она предполагает, что нам дан список смежности графа.

```

1 struct Edge {
2     int to;
3     int weight;
4 };
5
6 vector<vector<Edge>> graph; // undirected adjacency list
7
8 const int INF = 1e9;
9
10 vector<pair<int, int>> FindMST() {
11     int n = graph.size();
12
13     vector<bool> used(n, false);
14     vector<int> min_edge(n, INF);
15     vector<int> best_edge(n, -1);
16
17     vector<pair<int, int>> mst_edges;
```

```

18
19 min_edge[0] = 0;
20
21 for (int i = 0; i < n; ++i) {
22     int u = -1;
23
24     for (int v = 0; v < n; ++v)
25         if (!used[v] && (u == -1 || min_edge[v] < min_edge[u]))
26             u = v;
27
28     if (u == -1 || min_edge[u] == INF) {
29         // graph is unconnected
30         return {};
31     }
32
33     used[u] = true;
34
35     if (u != 0) {
36         mst_edges.emplace_back(u, best_edge[u]);
37     }
38
39     for (auto [v, w] : graph[u]) {
40         if (!used[v] && w < min_edge[v]) {
41             min_edge[v] = w;
42             best_edge[v] = u;
43         }
44     }
45 }
46
47 return mst_edges;
48 }

```

Псевдокод (приоритетная очередь). Приведём реализацию алгоритма Прима за $O((|V|+|E|) \log |V|)$ на приоритетной очереди. В отличие от алгоритма Дейкстры, здесь нужно поддерживать массив `used`, так как необходимо следить за тем, что новое ребро должно идти из уже построенного остова наружу.

```

1 struct Edge {
2     int to;
3     int weight;
4 };
5
6 vector<vector<Edge>> graph; // undirected adjacency list
7
8 const int INF = 1e9;
9
10 int FindMST() {
11     int n = graph.size();
12
13     vector<bool> used(n, false);
14     vector<int> dist(n, INF);
15     priority_queue<pair<int, int>, vector<pair<int, int>, greater<>> pq;
16
17     dist[0] = 0;
18     pq.emplace(0, 0);
19
20     int mst_weight = 0;

```

```

21
22 while (!pq.empty()) {
23     auto [d, u] = pq.top();
24     pq.pop();
25
26     if (used[u]) {
27         continue;
28     }
29
30     used[u] = true;
31     mst_weight += d;
32
33     for (auto [v, w] : graph[u]) {
34         if (!used[v] && w < dist[v]) {
35             dist[v] = w;
36             pq.emplace(dist[v], v);
37         }
38     }
39 }
40
41 return mst_weight;
42 }

```

3 Алгоритм Краскала

Введение. Приведём другой алгоритм построения MST за $O(|E| \log |E|)$.

Алгоритм 3.1 (Краскала). Посмотрим на задачу под другим углом. Пусть теперь разрез графа состоит из нескольких попарно несвязных множеств связанных вершин $\langle S_1, S_2, \dots, S_k \rangle, \forall i \neq j \hookrightarrow S_i \cap S_j = \emptyset, S_1 \cup S_2 \cup \dots \cup S_k = V$. Для каждой пары этих множеств всё ещё выполнена лемма о безопасном ребре. Тогда можно выбрать ребро минимального веса между всеми парами рёбер и добавить его в граф. По лемме о безопасном ребре это ребро будет принадлежать одному из MST. Поэтому алгоритм Краскала следующий:

1. Отсортируем рёбра по весу;
2. Итерируясь по рёбрам (по увеличению веса), проверяем, приводит ли его добавление к циклу. Если не приводит, то берём, иначе — пропускаем.

Алгоритм Краскала — это то же применение леммы о безопасном ребре, что и в алгоритме Прима. Заметим, что сортировка рёбер по весу происходит за $O(|E| \log |E|)$, а проверку на образование цикла можно поддерживать в системе непересекающихся множеств. Тогда общее время работы алгоритма составит $O(|E| \log |E|) + O(|E| \cdot \alpha(|V|)) = O(|E| \log |E|)$.

Псевдокод. Представим реализацию алгоритма Краскала с помощью СНМ.

```

1 struct DSU {
2     vector<int> p, sz;
3
4     DSU(int n) : p(n), sz(n, 1) {
5         iota(all(p), 0);
6     }
7
8     int Get(int v) {
9         if (p[v] == v) return v;

```

```

10     return p[v] = Get(p[v]);
11 }
12
13 void Unite(int u, int v) {
14     u = Get(u), v = Get(v);
15     if (u == v) return;
16     if (sz[u] > sz[v]) swap(u, v);
17     sz[v] += sz[u];
18     p[u] = v;
19 }
20 };
21
22 struct Edge {
23     int u, v, w;
24 };
25
26 bool cmp(Edge& a, Edge& b) {
27     return a.w < b.w;
28 }
29
30 vector<Edge> FindMST(int graph_size, vector<Edge>& edges) {
31     vector<Edge> mst;
32     sort(all(edges), cmp);
33     DSU snm(graph_size);
34     for (Edge& i : edges) {
35         if (snm.Get(i.u) != snm.Get(i.v)) {
36             mst.emplace_back(i);
37             snm.Unite(i.u, i.v);
38         }
39     }
40     return mst;
41 }

```

4 Алгоритм Борувки

Введение. Разберём ещё один алгоритм поиска MST, который работает в худшем случае за $O((|V| + |E|) \log |V|)$, но в среднем работает быстрее. Он немного похож на алгоритм Краскала, но не сортирует рёбра, а на каждой итерации за линию объединяет пары компонент связности.

Алгоритм 4.1 (Борувки). Схема алгоритма:

1. Изначально каждая вершина графа G — тривиальное дерево, а рёбра не принадлежат никакому дереву.
2. Для каждого дерева T найдём ребро минимального веса, инцидентное T . Добавим все такие рёбра.
3. Повторяем шаг 2, пока в графе не останется только одно дерево T .

В случае, когда все рёбра различного веса, алгоритм описан. Если есть рёбра одинакового веса, алгоритм работает неправильно. Например, на графе K_3 (треугольник) с весами рёбер 1. На первой же итерации в T могут быть добавлены все рёбра. Чтобы избежать этой проблемы, если рёбер минимального веса, инцидентных компоненте связности, несколько, выберем ребро с минимальным номером.

Лемма 4.2. Пусть C — некоторая компонента текущего леса, построенного алгоритмом. Пусть e — ребро минимального веса среди всех рёбер, инцидентных C . Тогда e является безопасным.

Доказательство. Действительно, рассмотрим разрез $\langle C, V \setminus C \rangle$. Ребро e пересекает этот разрез и имеет минимальный вес среди всех рёбер, пересекающих его. По лемме о безопасном ребре e является безопасным. Если минимальных рёбер несколько, то в алгоритме выбирается ребро с минимальным номером. Это лишь фиксирует однозначный выбор среди всех безопасных рёбер. $\square$

Утверждение 4.3. После добавления всех выбранных рёбер циклов не возникает.

Доказательство. Предположим противное: на некоторой итерации среди добавленных рёбер образовался цикл. Тогда все рёбра этого цикла соединяют разные текущие компоненты. Рассмотрим в этом цикле ребро с наименьшим номером среди рёбер максимального веса, обозначим его e . Это ребро было выбрано некоторой компонентой C . Но в цикле есть ещё одно ребро, тоже выходящее из C , и его вес не больше веса e (потому что e выбрано как ребро максимального веса в цикле). Если его вес строго меньше, то e вообще не могло быть минимальным инцидентным C . Если вес равен, то по правилу разрешения равенства должно было быть выбрано ребро с меньшим номером, а не e . Противоречие. Значит, циклов среди одновременно добавляемых рёбер нет. $\square$

Утверждение 4.4 (инвариант). После каждой итерации существует MST, содержащее все рёбра, уже добавленные алгоритмом.

Доказательство. Докажем утверждение по индукции.

База. До начала алгоритма рёбер нет, утверждение выполнено.

Шаг. Пусть после некоторой итерации инвариант верен. На следующей итерации для каждой компоненты выбирается безопасное ребро. По лемме каждое такое ребро можно добавить, не теряя возможности достроить до MST. Так как все выбранные рёбра вместе не образуют цикл, после их добавления снова получаем лес, содержащийся в некотором MST. $\square$

Теорема 4.5 (корректность). Алгоритм Борувки строит MST.

Доказательство. На каждой итерации каждая компонента выбирает ребро, выходящее в другую компоненту, значит каждая компонента объединяется хотя бы с одной другой. Поэтому число компонент уменьшается хотя бы вдвое. В частности, алгоритм обязательно завершится за конечное число шагов. Когда остаётся одна компонента, множество выбранных рёбер связно и ациклично, значит это остовное дерево. По инварианту это дерево содержится в некотором MST. Но остовное дерево не может строго содержаться в другом остовном дереве, следовательно само является MST. $\square$

Теорема 4.6 (время работы). Алгоритм Борувки в худшем случае работает за $O((|V| + |E|) \log |V|)$.

Доказательство. Одна итерация алгоритма выполняется за $O(|V| + |E|)$, всего итераций $O(\log |V|)$, потому что на каждой итерации число компонент уменьшается хотя бы вдвое. Поэтому итоговая сложность не превышает $O((|V| + |E|) \log |V|)$. $\square$

Псевдокод. Приведём немного ослабленную реализацию алгоритма Борувки за $O((|E| + |V|) \cdot \alpha(|V|) \cdot \log |V|)$, что асимптотически близко к $O(|E| \log |V|)$.

```

1 struct DSU {
2     vector<int> p, sz;
3
4     DSU(int n) : p(n), sz(n, 1) {
5         iota(all(p), 0);
6     }
7
8     int Get(int v) {
9         if (p[v] == v) return v;
10        return p[v] = Get(p[v]);
11    }

```

```

12
13 bool Unite(int u, int v) {
14     u = Get(u), v = Get(v);
15     if (u == v) return false;
16     if (sz[u] > sz[v]) swap(u, v);
17     sz[v] += sz[u];
18     p[u] = v;
19     return true;
20 }
21 };
22
23 struct Edge {
24     int u, v, w, id;
25 };
26
27 vector<Edge> FindMST(int n, vector<Edge>& edges) {
28
29     vector<Edge> mst;
30     int comp_count = n;
31
32     DSU snm(n);
33
34     while (comp_count > 1) {
35         vector<int> best(n, -1);
36         for (int i = 0; i < edges.size(); ++i) {
37             int pu = snm.Get(edges[i].u);
38             int pv = snm.Get(edges[i].v);
39             if (pu == pv) continue;
40
41             auto better = [&](int old_ind, int new_ind) {
42                 if (old_ind == -1) return true;
43                 if (edges[new_ind].w != edges[old_ind].w)
44                     return edges[new_ind].w < edges[old_ind].w;
45                 return edges[new_ind].id < edges[old_ind].id;
46             };
47
48             if (better(best[pu], i)) best[pu] = i;
49             if (better(best[pv], i)) best[pv] = i;
50         }
51
52         bool changed = false;
53
54         for (int c = 0; c < n; ++c) {
55             if (best[c] == -1) continue;
56
57             Edge& e = edges[best[c]];
58
59             if (snm.Unite(e.u, e.v)) {
60                 mst.emplace_back(e);
61                 --comp_count;
62                 changed = true;
63             }
64         }
65
66         if (!changed) {
67             // graph is unconnected

```



```

68     return {};
69 }
70 }
71
72 return mst;
73 }

```

Заключение. Алгоритм Борувки на самом деле полезен. Его обычно не имеет смысла использовать, если задача — только найти MST в графе. Главное свойство алгоритма заключается в том, что за одну фазу число компонент уменьшается хотя бы вдвое. Это очень сильное свойство, в более сложных алгоритмах его можно использовать, чтобы сильно уменьшить граф и запустить алгоритм на меньшем числе вершин. То есть алгоритм Борувки можно использовать как шаг редукции к меньшему графу. Так поступают в вероятностном алгоритме поиска MST за $O(|V| + |E|)$ в среднем: сначала выполняют несколько шагов алгоритма Борувки, чтобы размер графа уменьшился, затем рекурсивно ищут минимальный остовный лес на полученном графе. Этот алгоритм не входит в программу курса.

5 Центроидная декомпозиция

Введение. Во многих задачах на деревьях требуется эффективно обрабатывать пути, расстояния или запросы между вершинами. Обычные обходы дерева часто оказываются слишком медленными, если такие действия нужно выполнять многократно. Для решения подобных задач используют специальные методы декомпозиции дерева на более простые части. Один из таких методов — центроидная декомпозиция.

Определение 5.1. *Центроид* неориентированного дерева — это вершина, удаление которой разобьёт дерево на компоненты, каждая из которых содержит не более $\frac{N}{2}$ вершин.

Алгоритм 5.2 (поиск центроида в дереве). Пусть дано дерево $G = (V, E)$. Подвесим дерево за произвольную вершину и для каждой вершины посчитаем размер поддерева $sz[v]$. Будем спускаться от корня, пока есть ребёнок с поддеревом размера больше $\frac{N}{2}$. Вершина v , в которой процесс остановится, является центроидом дерева.

Псевдокод.

```

1  int n;
2  vector<vector<int>> g; // adjacency list
3  vector<int> sz;
4
5  void FillSz(int v, int par = -1) {
6      sz[v] = 1;
7      for (int to : g[v]) {
8          if (to != par) {
9              FillSz(to, v);
10             sz[v] += sz[to];
11         }
12     }
13 }
14
15 int GetCentroid(int v, int par = -1) {
16     for (int to : g[v]) {
17         if (to != par && sz[to] > n / 2) {
18             return GetCentroid(to, v);
19         }
20     }

```

```

21     return v;
22 }
23
24 signed main() {
25     // read graph
26
27     sz.resize(n);
28     FillSz(0);
29     int centroid = GetCentroid(0);
30 }

```

Теорема 5.3 (корректность). В произвольном неориентированном дереве всегда есть центроид. Приведённый алгоритм корректно находит центроид в произвольном неориентированном дереве T .

Доказательство. Алгоритм остановится в некоторой вершине, потому что алгоритм DFS спускается от корня к листу, у которого размер поддеревья равен 1. Пусть алгоритм остановился в вершине v , которая имеет поддеревья $T_1, T_2, \dots, T_k$. Поскольку рекурсивный вызов не произошёл, верны следующие неравенства:

$$\begin{cases} |T_1| \leq \frac{N}{2} \\ |T_2| \leq \frac{N}{2} \\ \vdots \\ |T_k| \leq \frac{N}{2} \end{cases}$$

После удаления вершины v дерево распадётся на компоненты $T_1, T_2, \dots, T_k$ и P , где P — компонента родителя v . Тогда

$$|P| + 1 + \sum_{i=1}^k |T_i| = N \implies |P| = N - 1 - \sum_{i=1}^k |T_i|$$

С другой стороны, поскольку алгоритм спустился в вершину v , верно следующее:

$$1 + \sum_{i=1}^k |T_i| > \frac{N}{2} \implies |P| = N - \left(1 + \sum_{i=1}^k |T_i|\right) < \frac{N}{2}$$

Таким образом,

$$\begin{cases} |T_1| \leq \frac{N}{2} \\ |T_2| \leq \frac{N}{2} \\ \vdots \\ |T_k| \leq \frac{N}{2} \\ |P| < \frac{N}{2} \end{cases}$$

Откуда v — центроид по определению. □

Теорема 5.4. В любом дереве существует не более двух центроидов. Причём если их два, то они смежны.

Доказательство. Мы уже доказали, что хотя бы один центроид всегда существует. Пусть теперь в графе есть два центроида, покажем, что тогда они смежны.

Докажем утверждение от противного. Пусть c_1 и c_2 — несмежные центроиды дерева. Подвесим дерево за эти два центроида как показано на рисунке.

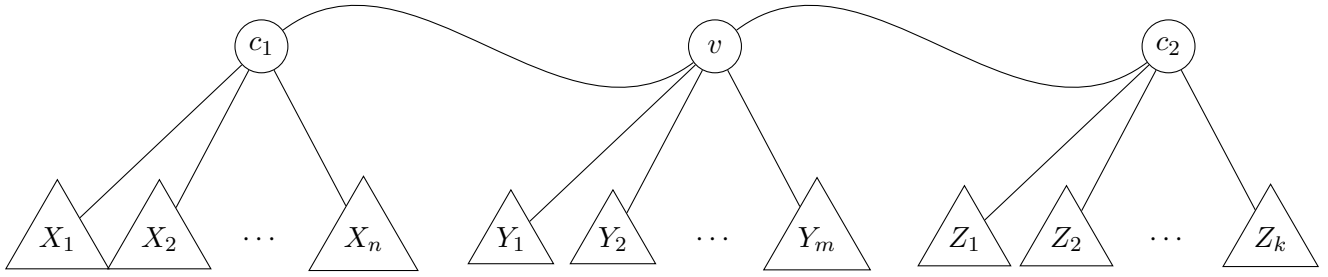


Рис. 1: Иллюстрация доказательства смежности центроидов

Поскольку c_1 не смежен с c_2 , найдётся другая вершина v на пути $c_1 \rightsquigarrow c_2$. Обозначим поддеревья c_1 как $X_1, X_2, \dots, X_n$, поддеревья v как $Y_1, Y_2, \dots, Y_m$, поддеревья c_2 как $Z_1, Z_2, \dots, Z_k$. Поскольку c_1 — центроид, для него выполнено:

$$\begin{cases} |X_1| \leq \frac{|V|}{2} \\ |X_2| \leq \frac{|V|}{2} \\ \vdots \\ |X_n| \leq \frac{|V|}{2} \\ 2 + \sum_{i=1}^m |Y_i| + \sum_{i=1}^k |Z_i| \leq \frac{|V|}{2} \end{cases}$$

Поскольку c_2 — центроид, для него выполнено:

$$\begin{cases} |Z_1| \leq \frac{|V|}{2} \\ |Z_2| \leq \frac{|V|}{2} \\ \vdots \\ |Z_k| \leq \frac{|V|}{2} \\ 2 + \sum_{i=1}^n |X_i| + \sum_{i=1}^m |Y_i| \leq \frac{|V|}{2} \end{cases}$$

Кроме того, $3 + \sum_{i=1}^n |X_i| + \sum_{i=1}^m |Y_i| + \sum_{i=1}^k |Z_i| = |V|$. В итоге имеем систему:

$$\begin{cases} |X_i| \leq \frac{|V|}{2}, \quad \forall i \in \{1, 2, \dots, n\} \\ |Z_i| \leq \frac{|V|}{2}, \quad \forall i \in \{1, 2, \dots, k\} \\ 2 + \sum_{i=1}^m |Y_i| + \sum_{i=1}^k |Z_i| \leq \frac{|V|}{2} \\ 2 + \sum_{i=1}^n |X_i| + \sum_{i=1}^m |Y_i| \leq \frac{|V|}{2} \\ 3 + \sum_{i=1}^n |X_i| + \sum_{i=1}^m |Y_i| + \sum_{i=1}^k |Z_i| = |V| \end{cases}$$

Обозначим:

$$A = \sum_{i=1}^n |X_i|, \quad B = \sum_{i=1}^m |Y_i|, \quad C = \sum_{i=1}^k |Z_i|$$

Тогда система переписывается так:

$$\begin{cases} A \geq 0, B \geq 0, C \geq 0 \\ 2 + B + C \leq \frac{|V|}{2} \\ 2 + A + B \leq \frac{|V|}{2} \\ 3 + A + B + C = |V| \end{cases}$$

Сложим второе и третье, получим $4 + A + 2B + C \leq |V|$. Подставим $|V|$ из четвёртого:

$$4 + A + 2B + C \leq 3 + A + B + C$$

Получаем, что $B \leq -1$, но $B = \sum_{i=1}^m |Y_i|$. Противоречие, следовательно, система неразрешима в неотрицательных целых числах. Следовательно, центроиды в дереве смежны.

Покажем теперь, что в дереве не может быть больше двух центроидов. Действительно, пусть в дереве есть центроиды c_1, c_2, c_3 . Тогда, поскольку все центроиды попарно смежны, должны быть рёбра $(c_1, c_2), (c_2, c_3), (c_1, c_3)$. Получаем цикл $c_1 \rightarrow c_2 \rightarrow c_3 \rightarrow c_1$ — противоречие с определением дерева. Следовательно, в произвольном дереве один или два центроида, причём если их два, то они смежны. □

Определение 5.5. Центроидной декомпозицией дерева T называют дерево $CD(T)$, построенное по следующим правилам:

Корнем $CD(T)$ выбирается вершина c — центроид T ;

Детьми корня $CD(T)$ становятся деревья $CD(T_1), \dots, CD(T_k)$, где $T_1, \dots, T_k$ — компоненты после удаления c из T .

Пример 5.6. Приведём пример центроидной декомпозиции дерева.

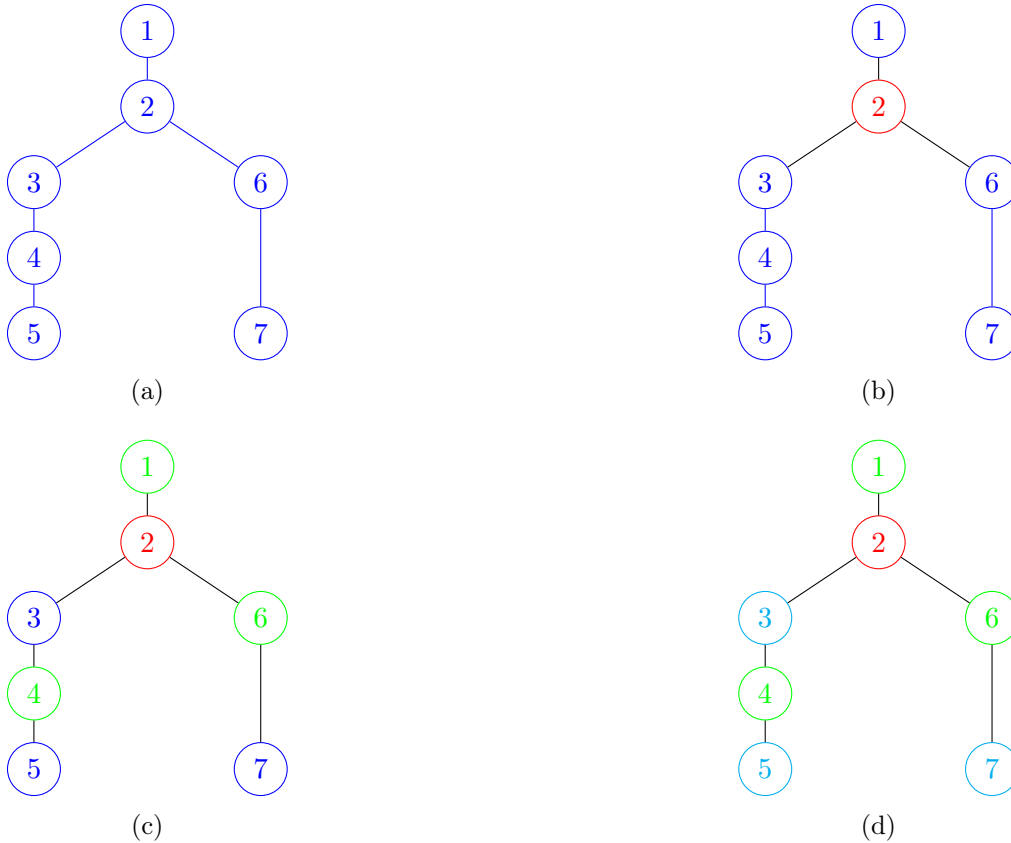
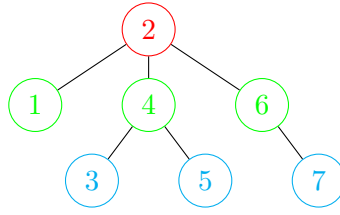


Рис. 2: Визуализация центроидной декомпозиции

Рис. 3: Полученное дерево $CD(T)$

Здесь синим обозначены компоненты, для которых центроид ещё не найден. Красным обозначен центроид первого уровня, зелёным — центроиды второго уровня, циановым — центроиды третьего уровня.

Теорема 5.7. Пусть дано дерево $T = (V, E)$, тогда глубина дерева $CD(T)$ равна $O(\log |V|)$.

Доказательство. Глубина дерева $CD(T)$ — это максимальная длина пути от корня до листа. Рассмотрим путь от корня к листу максимальной длины в $CD(T)$: $c_0 \rightarrow c_1 \rightarrow c_2 \rightarrow \dots \rightarrow c_k$. Каждая вершина c_i — это центроид некоторого поддерева T_i . Обозначим $n_i = |T_i|$. По определению центроида $n_{i+1} \leq \frac{n_i}{2}$. Поскольку c_0 — корень $CD(T)$, то $|T_0| = |V|$. Имеем:

$$\begin{cases} n_0 = |V| \\ n_1 \leq \frac{|V|}{2} \\ n_2 \leq \frac{|V|}{4} \\ \vdots \\ n_k \leq \frac{|V|}{2^k} \end{cases}$$

Процесс поиска центроида продолжается, пока размер поддерева ≥ 1 , то есть $\frac{|V|}{2^k} \geq 1$, откуда $2^k \leq |V| \Rightarrow k \leq \log_2 |V|$. Следовательно, максимальная глубина $CD(T)$ равна $O(\log |V|)$. $\square$

Теорема 5.8. Пусть дано дерево $T = (V, E)$ и его центроидная декомпозиция $CD(T)$. Для любых вершин $u, v \in V$ обозначим $c = \text{LCA}_{CD(T)}(u, v)$ — наименьший общий предок u и v в дереве $CD(T)$. Тогда вершина c лежит на простом пути $u \rightsquigarrow v$ в исходном дереве T .

Доказательство. Рассмотрим вершину c . Если $u = c$ или $v = c$, то утверждение выполнено: конец пути $u \rightsquigarrow v$ сам лежит на этом пути. Пусть теперь $u \neq c$ и $v \neq c$. Так как c — наименьший общий предок u и v в $CD(T)$, то вершины u и v лежат в разных поддеревьях, вершины c в дереве $CD(T)$. По определению центроидной декомпозиции дети вершины c соответствуют компонентам связности дерева $T \setminus \{c\}$. Значит, u и v лежат в разных компонентах графа $T \setminus \{c\}$. Но в дереве единственный путь между двумя вершинами из разных компонент после удаления c обязан проходить через c . Иначе путь лежал бы в одной компоненте $T \setminus \{c\}$, что невозможно. Следовательно, вершина c принадлежит пути $u \rightsquigarrow v$ в дереве T . $\square$

Пример 5.9. Дано дерево T на n вершинах. Требуется поддерживать q запросов двух видов:

1. **Mark(v)** — пометить вершину v ,
2. **FindNearest(v)** — найти расстояние от v до ближайшей помеченной вершины.

С помощью центроидной декомпозиции эту задачу можно решить за время $O((n+q) \log n)$ и $O(n \log n)$ дополнительной памяти.

Для каждой вершины v храним список:

$$(c_0, d_0), (c_1, d_1), \dots$$

Где c_i — центроид на пути от v к корню $CD(T)$, а $d_i = \text{dist}(v, c_i)$. Для каждого центроида c поддерживаем $\text{best}[c] = \min \text{dist}(x, c)$ по всем активным x .

Рассмотрим запрос **FindNearest** для вершины v . Пусть ближайшая к ней помеченная вершина — u . Обозначим $c = \text{LCA}_{CD(T)}(u, v)$. Тогда c лежит на пути $u \rightsquigarrow v$, а также $\text{dist}(u, v) = \text{dist}(u, c) + \text{dist}(c, v)$ в исходном графе. По построению $\text{best}[c] = \text{dist}(u, c)$, следовательно, $\text{dist}(v, c) + \text{best}[c] = \text{dist}(v, c) + \text{dist}(c, u) = \text{dist}(v, u)$. Поэтому достаточно пройтись по всем предкам v в дереве $CD(T)$ и найти минимум по всем $\text{dist}(v, c) + \text{best}[c]$.

Чтобы обработать запрос **Mark(v)**, пройдемся по всем предкам v в дереве $CD(T)$ и обновим $\text{best}[c] \leftarrow \min(\text{best}[c], \text{dist}(c, v))$.

Псевдокод. Покажем, как реализовать решение этой задачи.

```

1 int n; // graph size
2 vector<vector<int>> g; // adjacency list
3 vector<int> sz;
4 vector<bool> used;
5 vector<int> centroid_parent; // CD(T)
6 vector<vector<pair<int, int>>> dist_to_centroid; // dist[v] = list {c, dist(v,
   c)};
7 vector<int> best;
8
9 void FillSz(int v, int p) {
10     sz[v] = 1;
11     for (int to : g[v]) {
12         if (to != p && !used[to]) {
13             FillSz(to, v);
14             sz[v] += sz[to];
15         }
16     }
17 }
18
19 int GetCentroid(int v, int p, int comp_size) {
20     for (int to : g[v]) {
21         if (to != p && !used[to] && sz[to] > comp_size / 2) {
22             return GetCentroid(to, v, comp_size);
23         }
24     }
25     return v;
26 }
27
28 void FillDist(int v, int p, int c, int d) {
29     dist_to_centroid[v].emplace_back(c, d);
30     for (int to : g[v]) {
31         if (to != p && !used[to]) {
32             FillDist(to, v, c, d + 1);
33         }
34     }
35 }
36
37 void BuildCD(int v, int p) {
38     FillSz(v, -1);
39     int c = GetCentroid(v, -1, sz[v]);
40     centroid_parent[c] = p;
41     used[c] = true;
42
43     FillDist(c, -1, c, 0);

```

```

44
45     for (int to : g[c]) {
46         if (!used[to]) {
47             BuildCD(to, c);
48         }
49     }
50 }
51
52 void Update(int v) {
53     for (auto [c, d] : dist_to_centroid[v]) {
54         best[c] = min(best[c], d);
55     }
56 }
57
58 int Query(int v) {
59     int res = INT_MAX;
60     for (auto [c, d] : dist_to_centroid[v]) {
61         if (best[c] != INT_MAX) {
62             res = min(res, best[c] + d);
63         }
64     }
65     return res;
66 }
67
68 int main() {
69     cin >> n;
70     g.resize(n);
71     sz.resize(n);
72     used.resize(n, false);
73     centroid_parent.resize(n);
74     dist_to_centroid.resize(n);
75     best.resize(n);
76
77     // read graph
78
79     BuildCD(0, -1);
80
81     for (int i = 0; i < n; ++i) {
82         best[i] = INT_MAX;
83     }
84
85     int q; cin >> q;
86     while (q--) {
87         int type, v;
88         cin >> type >> v;
89         --v;
90         if (type == 1) {
91             Update(v);
92         } else {
93             cout << Query(v) << '\n';
94         }
95     }
96 }

```

Для центроидной декомпозиции являются шаблонными функции `FillSz`, `GetCentroid`, `BuildCD`. В функцию `BuildCD` передаётся вершина из текущей компоненты связности. Для компоненты связно-

сти выполнено: все вершины между собой связны и для каждой вершины выполнено `used = false`. Функция находит центроид в компоненте, вычисляет для него всю необходимую информацию для поддержания запросов (`FillDist`) и рекурсивно находит центроиды в компонентах связности после удаления центроида из дерева (после `used[c] = true`).

Заключение. Итак, центроидная декомпозиция позволяет свести работу с исходным деревом к работе с деревом центроидов небольшой высоты. Это даёт возможность хранить в каждом центроиде информацию о соответствующей части дерева и затем использовать её при ответах на запросы.

6 Heavy-light декомпозиция

Введение. Рассмотрим другую декомпозицию дерева, специализирующуюся на запросах на путях и изменениях на путях в дереве.

Постановка задачи (пример задачи на HLD). Дано неориентированное дерево $T = (V, E)$ на $|V| = n$ вершинах. На каждой вершине написан её вес. Требуется эффективно поддерживать q запросов двух типов:

1. Увеличить вес вершины $v \in V$ на x ;
2. Узнать максимальный вес вершины на простом пути $u \rightsquigarrow v$.

Если бы не было запросов первого типа, эту задачу можно было бы решать за $O((n + q) \log n)$ с помощью двоичных подъёмов.

Определение 6.1. Пусть дано подвешенное дерево T . Будем называть ребро $e = (u, v) \in E$ *тяжёлым*, если u — непосредственный предок v и

$$\text{size}[v] = \max_{i \in \text{children}(u)} \text{size}[i]$$

Если для одной вершины подходящих рёбер несколько, то выбирается одно любое. То есть *тяжёлое ребро* — это ребро из вершины к её наибольшему потомку (в наибольшее по размеру поддереву). Все остальные рёбра будем называть *лёгкими*.

Утверждение 6.2. Произвольное подвешенное дерево T разбивается на непересекающиеся пути из тяжёлых рёбер.

Доказательство. В каждую вершину входит не более одного тяжёлого ребра (из предка), и из каждой вершины исходит не более одного ребра — по определению тяжёлого ребра. $\square$

Определение 6.3. Пусть дано подвешенное дерево T . Назовём *блоком* либо лёгкое ребро, либо вертикальный путь из тяжёлых рёбер.

Определение 6.4. Описанное разбиение подвешенного дерева T на блоки называется *HLD* (*heavy-light decomposition*).

Утверждение 6.5. Пусть дано подвешенное дерево T на $|V| = n$ вершинах. Тогда на любом пути будет $O(\log n)$ блоков.

Доказательство. Подвесим дерево за произвольную вершину. Доказательство разбивается на две части отдельно для лёгких рёбер и непрерывных путей из тяжёлых рёбер:

- Рассмотрим произвольную вершину и будем идти по вертикальному пути снизу вверх. Каждый раз, когда мы переходим по лёгкому ребру, размер поддерева текущей вершины увеличивается хотя бы в 2 раза, потому что если вершина связана с родителем лёгким ребром, то у родителя есть другой ребёнок, поддерево которого не меньше текущего. При переходе по лёгкому ребру размер поддерева увеличивается как минимум вдвое, следовательно, на пути от произвольной вершины к корню $O(\log n)$ лёгких рёбер.
- Каждый переход между тяжёлыми путями осуществляется по лёгкому ребру, а их на пути от произвольной вершины к корню всего $O(\log n)$.

Таким образом, путь $u \rightsquigarrow v$ разбивается на пути $u \rightsquigarrow a, a \rightsquigarrow v$, где $a = \text{LCA}(u, v)$, на каждом из которых содержится $O(\log n)$ блоков. Следовательно, и на пути $u \rightsquigarrow v$ будет $O(\log n)$ блоков. $\square$

Следствие 6.6. Чтобы поддерживать запросы на пути $u \rightsquigarrow v$, рассмотрим блоки, которые пересекает путь $u \rightsquigarrow v$ и вычислим ответ из этих блоков. Таких блоков $O(\log n)$. Суть HLD заключается в том, чтобы рассматривать не $O(n)$ вершин на каждом запросе, а разбить пути в дереве на непрерывные блоки, в каждом из которых можно считать ответ быстрее, чем за линейное время.

Решение. Разберём решение задачи, приведённой в начале параграфа. Сначала представим код, потому что идея декомпозиции намного проще её реализации. Автор конспекта это понимает, а потому идёт навстречу читателю. Заодно вспомним, как писать дерево отрезков.

```

1 constexpr int INF = 2e9;
2
3 struct SegmentTree {
4     vector<int> st;
5     int tsize;
6
7     SegmentTree() = default;
8
9     void build(int n) {
10         tsize = pow(2, ceil(log2(n)));
11         st.resize(2 * tsize, -INF);
12         for (int i = tsize; i < tsize + n; ++i) st[i] = 0;
13         for (int i = tsize - 1; i > 0; --i) st[i] = max(st[2 * i], st[2 * i +
14             1]);
15     }
16
17     void build(vector<int> a) {
18         tsize = pow(2, ceil(log2(a.size())));
19         st.resize(2 * tsize, -INF);
20         for (int i = tsize; i < tsize + a.size(); ++i) st[i] = a[i - tsize];
21         for (int i = tsize - 1; i > 0; --i) st[i] = max(st[2 * i], st[2 * i +
22             1]);
23     }
24
25     int get_max(int v, int l, int r, int lq, int rq) {
26         if (rq < l || r < lq) return -INF;
27         if (lq <= l && r <= rq) return st[v];
28         int m = (l + r) / 2;
29         return max(get_max(2 * v, l, m, lq, rq), get_max(2 * v + 1, m + 1, r, lq
30             , rq));
31     }
32
33     int get(int lq, int rq) { return get_max(1, 0, tsize - 1, lq, rq); }

```

```

32     void upd(int ind, int x) { // a[ind] += x;
33         ind += tsize;
34         st[ind] += x;
35         ind /= 2;
36         while (ind > 0) {
37             st[ind] = max(st[2 * ind], st[2 * ind + 1]);
38             ind /= 2;
39         }
40     }
41 } st;
42
43 vector<vector<int>> g;
44 vector<int> sz, tin, par, euler, ind, h;
45
46 void dfs_sz(int v = 0, int p = -1) {
47     sz[v] = 1;
48     par[v] = p;
49     for (int& to : g[v]) {
50         if (to != p) {
51             dfs_sz(to, v);
52             sz[v] += sz[to];
53         }
54         if (g[v][0] == p || sz[to] > sz[g[v][0]])
55             swap(to, g[v][0]);
56     }
57 }
58
59 void dfs(int v = 0, int index_ = 0, int height = 0) {
60     tin[v] = euler.size();
61     ind[v] = index_;
62     h[v] = height;
63     euler.push_back(v);
64     if (!g[v].empty() && g[v][0] != par[v]) dfs(g[v][0], index_ + 1, height +
65         1);
66     for (int i = 1; i < g[v].size(); ++i) {
67         int to = g[v][i];
68         if (to != par[v]) dfs(to, 0, height + 1);
69     }
70 }
71
72 int get_lca(int u, int v) { // O(logn) ~M(logn)
73     int U = euler[tin[u] - ind[u]];
74     int V = euler[tin[v] - ind[v]];
75     if (U == V) return h[u] < h[v] ? u : v;
76     if (h[U] > h[V]) return get_lca(par[U], v);
77     return get_lca(u, par[V]);
78 }
79
80 int get_query(int u, int v) {
81     // it is expected that v is ancestor of u
82     int ans = -INF;
83     while (true) {
84         int new_ind = tin[u] - ind[u];
85         if (new_ind < tin[v]) {
86             ans = max(ans, st.get(tin[v], tin[u]));
87             return ans;

```

```

87     }
88     ans = max(ans, st.get(new_ind, tin[u]));
89     if (new_ind == tin[v]) return ans;
90     u = par[euler[new_ind]];
91 }
92 }
93
94 void solve() {
95     int n; cin >> n;
96     g.resize(n);
97     sz.resize(n, 0);
98     tin.resize(n);
99     par.resize(n);
100    ind.resize(n);
101    h.resize(n);
102    for (int i = 0; i < n - 1; ++i) {
103        int u, v; cin >> u >> v;
104        --u, --v;
105        g[u].push_back(v);
106        g[v].push_back(u);
107    }
108
109    dfs_sz();
110    dfs();
111    st.build(n);
112
113    int q; cin >> q;
114    while (q--) {
115        char c;
116        int u, v; cin >> c >> u >> v;
117        if (c == 'I') st.upd(tin[u - 1], v);
118        else {
119            int lca = get_lca(u - 1, v - 1);
120            cout << max(get_query(u - 1, lca), get_query(v - 1, lca)) << '\n';
121        }
122    }
123 }

```

Разберём подробно эту реализацию. Для работы с графом нам понадобятся следующие сущности:

- `g` — список смежности,
- `sz` — размеры поддеревьев (не имеет значения, за какую вершину подвесить дерево, в приведённой реализации дерево подвешивается за вершину 0),
- `euler` — порядок обхода вершин,
- `tin[v]` — позиция, на которой вершина v впервые встречается в `euler`,
- `par[v]` — непосредственный предок v . В частности, `par[root] = -1`,
- `ind[v]` — порядковый номер вершины v в её тяжёлом пути $(0, 1, 2, \dots)$. Это нужно, чтобы было выполнено `euler[tin[v] - ind[v]]` — самая верхняя вершина в соответствующем тяжёлом пути для быстрого перехода внутри блока,
- `h[v]` — высота (глубина) вершины в графе (от корня). В частности, `h[0] = 0`.

Разберём построение размерной динамики HLD.

```

1 void dfs_sz(int v = 0, int p = -1) {
2     sz[v] = 1;
3     par[v] = p;
4     for (int& to : g[v]) {
5         if (to != p) {
6             dfs_sz(to, v);
7             sz[v] += sz[to];
8         }
9         if (g[v][0] == p || sz[to] > sz[g[v][0]])
10             swap(to, g[v][0]);
11     }
12 }

```

По умолчанию запуск происходит из корня (вершина 0) с предком (-1) . Тогда $sz[v] = 1 + \sum_{i \in \text{children}(v)} sz[i]$. Здесь всё так же, как и в обычном dfs на дереве, но есть изменение: вершину с наибольшим sz мы поставим на первое место в списке $g[v]$. Это упростит нам написание второго dfs для заполнения глобальных полей HLD. Обратите внимание: $\text{swap}(to, g[v][0])$ корректен, так как to — это ссылка на вершину-потомка v .

Разберём второй dfs с заполнением HLD.

```

1 void dfs(int v = 0, int index_ = 0, int height = 0) {
2     tin[v] = euler.size();
3     ind[v] = index_;
4     h[v] = height;
5     euler.push_back(v);
6     if (!g[v].empty() && g[v][0] != par[v]) dfs(g[v][0], index_ + 1, height + 1);
7     for (int i = 1; i < g[v].size(); ++i) {
8         int to = g[v][i];
9         if (to != par[v]) dfs(to, 0, height + 1);
10    }
11 }

```

Для вершины v передаём $index_$ — индекс этой вершины в тяжёлом пути и $height$ — глобальная высота в дереве. Сначала отдельно обработаем ребро в тяжёлую вершину. Не забываем про важные условия, если вершина висячая или дерево из одной вершины. Далее — стандартный обход, аналогичный обычному dfs.

Покажем изящный метод поиска LCA в получившемся графе.

```

1 int get_lca(int u, int v) { //  $O(\log n)$  ~  $M(\log n)$ 
2     int U = euler[tin[u] - ind[u]];
3     int V = euler[tin[v] - ind[v]];
4     if (U == V) return h[u] < h[v] ? u : v;
5     if (h[U] > h[V]) return get_lca(par[U], v);
6     return get_lca(u, par[V]);
7 }

```

Реализация будет рекурсивной! Можно было бы написать LCA на sparse table, но поскольку асимптотика запроса не меньше, чем асимптотика поиска LCA, мы ничего не потеряем асимптотически.

Предки-начала пути вершин u, v равны $U = \text{euler}[\text{tin}[u] - \text{ind}[u]]$, $V = \text{euler}[\text{tin}[v] - \text{ind}[v]]$ — из свойства обхода. Где будет находиться $\text{LCA}(u, v)$? Возможны два случая:

1. Вершины u, v имеют общего предка-начало их тяжёлого пути. Тогда у вершины V ребро $p[V]$ лёгкое, значит, $\text{LCA}(u, v)$ — это та вершина, которая выше, то есть та, у которой h меньше.

2. Вершины u, v имеют разных предков-начала их тяжёлых путей. Тогда логично, что та вершина, которая выше, не всегда может являться $LCA(u, v)$. Поэтому мы запускаем в этом случае поиск для предка нижней вершины-начала пути и остальной вершины без изменений.

Такая реализация поиска $LCA(u, v)$ занимает $O(\log n)$ времени и $O(\log n)$ дополнительной памяти (из-за рекурсии).

Разберём структуру функции ответа на запрос в общем случае для HLD.

```

1 int get_query(int u, int v) {
2     // it is expected that v is ancestor of u
3     int ans = -INF;
4     while (true) {
5         int new_ind = tin[u] - ind[u];
6         if (new_ind < tin[v]) {
7             ans = max(ans, st.get(tin[v], tin[u]));
8             return ans;
9         }
10        ans = max(ans, st.get(new_ind, tin[u]));
11        if (new_ind == tin[v]) return ans;
12        u = par[euler[new_ind]];
13    }
14 }
```

Положим, что v — предок u . В этой функции используется ещё одно свойство обхода, который мы задали.

Утверждение 6.7. Все тяжёлые пути в такой реализации лежат подряд в получившемся вершинном обходе.

Доказательство. Поскольку мы каждый раз при вызове `dfs` посещали сначала вершину-продолжение текущего тяжёлого пути, то все пути в эйлеровом обходе будут находиться подряд. $\square$

Это свойство важно, поскольку если на вершинах написаны числа, то в эйлеровом обходе на очередном тяжёлом пути эти числа будут лежать подряд (образовывать блок), и их можно эффективно пересчитать, например, деревом отрезков или префиксными суммами.

В этой функции очень легко ошибиться, поэтому разберём её детально. Изначально зададим `ans = -INF` — максимум среди всех чисел на пути от u до v , где v — предок u . Далее цикл, в котором мы будем перемещать u к v так, как описано выше. Тогда `new_ind = tin[u] - ind[u]` — индекс в массиве `euler` элемента-начала тяжёлого пути u . Тогда возможны два случая:

1. Мы перепрыгнули v , то есть u и v находились в одном пути. Тогда мы обновляем ответ: `ans = max(ans, st.get(tin[v], tin[u]))` и возвращаем значение.
2. u и v находились в разных блоках. Тогда обновляем ответ: `ans = max(ans, st.get(new_ind, tin[u]))` и переходим к следующему тяжёлому пути, то есть `u = par[euler[new_ind]]`.

Для поддержки операций изменения в вершине, то в `st` изменим значение на индексе `tin[v]`. Тогда дерево отрезков будет синхронизировано с работой `euler`.

Заключение. HLD — мощный алгоритм для задач с запросами на путях между конкретными вершинами.